CNN Compression

Sparsh Mittal

Basics Related to Compression
1. Sparse Matrix Storage Formats
2. Huffman Encoding

Sparsh Mittal

Formats for Storing Sparse Matrices

Formats for Storing Sparse Matrices

- Coordinate format (COO) (Row column value).
- Compressed sparse row (CSR).
- Compressed sparse column (CSC).

Row-Column-Value Representation

$$\begin{bmatrix} 0 & 0 & 3 & 0 & 4 \\ 0 & 0 & 5 & 7 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 6 & 0 & 0 \end{bmatrix}$$

Sparse Matrix

Row	0	0	1	1	3	3
Column	2	4	2	3	1	2
Value	3	4	5	7	2	6

Compressed Sparse Row Format

It requires $2a+n+1$ numbers, where

- a=number of nonzero elements
- n=number of rows or columns.

Compressed Sparse Row Format

- Array A stores nonzero entries in row-major order
- IA is defined recursively: IA[0] = 0
- IA[i] = IA[i−1] + (number of nonzero elements on the (i−1)th row in the original matrix)
- JA contains the column index of each non-zero element and hence is of length a.

$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 5 & 8 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 6 & 0 & 0 \end{pmatrix}$$

Original matrix

A = [5 8 3 6]
Row offset IA = [0 0 2 3 4]
Column JA = [0 1 2 1]

CSR representation

Example

Find CSR representation of this matrix

0	5	2	5	0	0
0	0	0	7	0	0
2	3	0	0	4	2
8	4	0	0	0	0
0	0	0	0	8	3
3	2	0	0	0	0

A = [5 2 5 7 2 3 4 2 8 4 8 3 3 2]

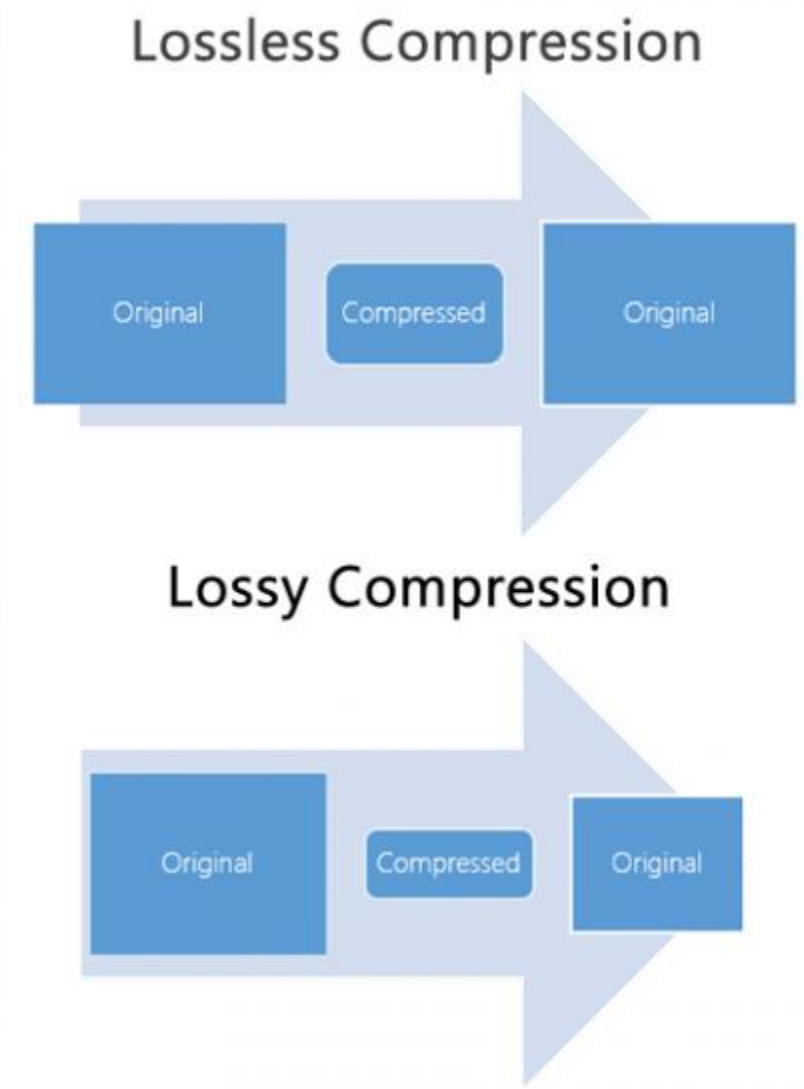
JA = [1 2 3 3 0 1 4 5 0 1 4 5 0 1]

IA = [0 3 4 8 10 12 14]

Huffman Encoding

Huffman Encoding

- A Huffman code is an optimal prefix code used for lossless data compression.
- It uses variable-length codewords to encode source symbols.
- The table is derived from the occurrence probability for each symbol.
- More common symbols are represented with fewer bits.



Characteristics of Prefix Codes

- The variable-length codes are prefix code, that is, they are assigned in such a way that the code assigned to one character is not prefix of code assigned to any other character.
- This ensures that there is no ambiguity when decoding the generated bit stream.

Example on prefix-code

Given the following coding:

character	encoding
a	0
b	111
c	1011
d	1010
r	110
!	100

Decode the message 011111001011010100111110010 0

Answer: abracadabra!

Explanation:

|0|111|110|0|1011|0|1010|0|111|110|0|100

a b r a c a d a b r a !

Examples on prefix-code

Example 1: [1,10,110,111] is NOT a prefix code, because (1) is a prefix of 10, 110 and 111.

Example 2: [0,10,110,111] is a prefix code, because no code is a prefix of another code.

Example 3: Assume our codewords are [1, 2, 33, 34, 50, 61].

Assume that we receive a message 1611333425012, but we don't know where one codeword starts or ends.

By prefix code property, we can decode the message as consisting of 1 61 1 33 34 2 50 1 2

Example 1 on Fixed-Length encoding

Character	Frequency
a	5
b	9
c	12
d	13
e	16
f	45

Character	Codeword
a	000
b	001
c	010
d	011
e	100
f	101

Total storage overhead = 100*3= 300 bits

Example 2 on Huffman encoding

Character	Frequency
a	5
b	9
c	12
d	13
e	16
f	45

Character	Code word
f	0
c	100
d	101
a	1100
b	1101
e	111

Total storage overhead = 5*4+9*4+12*3+13*3+16*3+45*1 = 224 bits

Lesson: When frequency distribution is skewed, Huffman encoding is advantageous than binary encoding.

Example 3 on Huffman encoding

Character	Frequency
a	20
b	19
c	12
d	13
e	16
f	20

Character	Code word
f	0
c	100
d	101
a	1100
b	1101
e	111

Total storage overhead = 20*4+19*4+12*3+13*3+16*3+20*1 = 299 bits

Lesson: When frequency distribution is not skewed, Huffman encoding is not advantageous than binary encoding.